



PROGRAMAÇÃO ORIENTADA A OBJETOS

Prédio 1 – Sala 408

AULA 05 – (19:00-20:20) e (20:40 – 22:00)

Quinta-feira 26/03/26

Prog260326a - fonte

```
1 # 1. Definição da Classe (O Molde/Projeto)
```

```
2 class Carro:
```

```
3     def __init__(self, marca, modelo):
```

```
4         self.marca = marca           # Atributo
```

```
5         self.modelo = modelo        # Atributo
```

```
6         self.ligado = False         # Atributo com valor inicial
```

```
7
```

```
8     def ligar(self):
```

```
9         self.ligado = True
```

```
10        print(f"O {self.modelo} está ligado! Vrummm!")
```

```
11
```

```
12 # 2. Programa Principal (Uso da Classe)
```

```
13 if __name__ == "__main__":
```

```
14     # Instanciando objetos (Criando os carros reais)
```

```
15     meu_carro = Carro("Toyota", "Corolla")
```

```
16     carro_do_professor = Carro("Ford", "Mustang")
```

```
17
```

```
18     # Acessando dados e executando ações
```

```
19     print(f"Meu carro é um {meu_carro.modelo}")
```

```
20     carro_do_professor.ligar()
```

Chamado quando um objeto é instanciado.

Método que define o comportamento do objeto. O programador deve chama-lo no código.

Instanciando o objeto meu_carro

Instanciando o objeto carro_do_professor

Prog260326a - executado

```
Meu carro é um Corolla  
O Mustang está ligado! Vrummm!
```

ABSTRAÇÃO

Prog260326b – fonte (01-22)

```
1 # abc: biblioteca Abstract Base Classes
2 # ABC: Superclasse. A classe que herda dela não pode ter objeto instanciado
3 # abstractmethod:
4 # Método decorador identificado pelo símbolo @
5 # Carimba os métodos que as classes filhas devem escrever
6 from abc import ABC, abstractmethod
7 import math
8
9 # 1. A CLASSE ABSTRATA (O Conceito Geral)
10 class Forma(ABC):
11     @abstractmethod
12     def calcular_area(self):
13         """Este método é um contrato: toda forma DEVE saber calcular sua área."""
14         pass
15
16 # 2. CLASSES CONCRETAS (A Implementação Real)
17 class Quadrado(Forma):
18     def __init__(self, lado):
19         self.lado = lado
20
21     def calcular_area(self):
22         return self.lado ** 2
```

ABSTRAÇÃO

Prog260326b – fonte (24-43)

```
24 class Circulo(Forma):
25     def __init__(self, raio):
26         self.raio = raio
27
28     def calcular_area(self):
29         return math.pi * (self.raio ** 2)
30
31 # 3. PROGRAMA PRINCIPAL
32 if __name__ == "__main__":
33     # Criando instâncias das formas
34     formas = [
35         Quadrado(lado=4),
36         Circulo(raio=3)
37     ]
38
39     print("--- Relatório de Áreas ---")
40     for f in formas:
41         # O programa não sabe se é quadrado ou círculo,
42         # ele apenas confia que a 'Forma' sabe calcular_area()
43         print(f"Área da forma: {f.calcular_area():.2f}")
```

ABSTRAÇÃO

Prog260326b – executando

```
--- Relatório de Áreas ---  
Área da forma: 16.00  
Área da forma: 28.27
```

HERANÇA

Prog260326c – fonte (01-19)

```
1 # 1. SUPERCLASSE (A base comum para todos)
2 class Funcionario:
3     def __init__(self, nome, salario):
4         self.nome = nome
5         self.salario = salario
6
7     def exibir_dados(self):
8         print(f"Nome: {self.nome} | Salário: R$ {self.salario:.2f}")
9
10 # 2. SUBCLASSE (Especialização do funcionário)
11 class Gerente(Funcionario): # <--- Indica que Gerente HERDA de Funcionario
12     def __init__(self, nome, salario, bonus):
13         # O super() chama o construtor da Superclasse para preencher nome e salario
14         super().__init__(nome, salario)
15         self.bonus = bonus
16
17     def calcular_total(self):
18         total = self.salario + self.bonus
19         print(f"Total com bônus: R$ {total:.2f}")
```

HERANÇA

Prog260326c – fonte (21-29)

```
21 # 3. PROGRAMA PRINCIPAL
22 if __name__ == "__main__":
23     # Criando um objeto da Subclasse
24     g1 = Gerente("Antonio", 5000, 1500)
25
26     # O Gerente consegue usar métodos da Superclasse (exibir_dados)
27     # e métodos próprios (calcular_total)
28     g1.exibir_dados()
29     g1.calcular_total()
```


HERANÇA

Prog260326c – executando

```
Nome: Antonio | Salário: R$ 5000.00  
Total com bônus: R$ 6500.00
```

HERANÇA MÚLTIPLA

Prog260326d – fonte (01-17)

```
1 # 1. Primeira Superclasse
2 class Relogio:
3     def mostrar_hora(self):
4         return "14:30:00"
5
6 # 2. Segunda Superclasse
7 class Calendario:
8     def mostrar_data(self):
9         return "26/03/2026"
10
11 # 3. Subclasse que herda de AMBAS (Herança Múltipla)
12 class Smartwatch(Relogio, Calendario):
13     def mostrar_tudo(self):
14         # Ela tem acesso aos métodos de ambas as "mães"
15         hora = self.mostrar_hora()
16         data = self.mostrar_data()
17         print(f"Smartwatch - Data: {data} | Hora: {hora}")
```

HERANÇA MÚLTIPLA

Prog260326d – fonte (19-22)

```
19 # 4. Programa Principal
20 if __name__ == "__main__":
21     meu_relogio = Smartwatch()
22     meu_relogio.mostrar tudo()
```

HERANÇA MÚLTIPLA

Prog260326d – executando

| Smartwatch – Data: 26/03/2026 | Hora: 14:30:00

TAREFAS

Modificar o programa 260326a para que tenha um método desligar. Chamar todos os métodos de cada objeto que foi instanciado (meu_carro e carro_do_professor).

Modificar o programa 260326b para que tenha uma classe chamada Equilatero com um método para calcular a sua área. Instanciar esta classe junto com as outras.

Modificar o programa 260326c para que tenha uma classe chamada Minion que tenha os mesmos métodos da classe Gerente. Instanciar e usar esta classe no programa principal.

Modificar o programa 260326d para que o programa retorne a data e hora atuais (procurar os comandos para isso)

